π0.5 分拣螺丝：Jetson Orin / Thor OpenPI-Torch 边缘加速实施手册

权重：`pi05_policy_screw_sorting_single_sft_ep279_v30_annotated-pi07`，step 030000。
入口：OpenPI / LeRobot 的 PyTorch `model.safetensors`。按阶段执行，上一阶段门禁没过就停。

1. 先选板、先定配置

这份策略在工作站 RTX 3090（SM 8.6）上继续用现有 `torch.compile`。边缘侧只使用下面两块板。两块板都读同一份 safetensors，都把键名里的 `model.` 前缀自动剥掉，都不需要先把权重转成另一份文件。

	Jetson AGX Orin	**Jetson AGX Thor**
计算能力	SM 8.7	SM 11.0
系统	JetPack 6.x，CUDA 12.6	JetPack 7.2，CUDA 13
前端类	`Pi05TorchFrontendRtx`	`Pi05TorchFrontendThor`
编译架构	`-DGPU_ARCH=87`	`-DGPU_ARCH=110`，并打开 `FLASHRT_ENABLE_PI05_THOR`
动作长度	构造参数 `chunk_size=32`	源码里写死为 10。落地前改成 32 再编译
能用的量化	BF16；W8A8（INT8）	BF16；FP8；NVFP4
开不了的	FP8、NVFP4、W4A8	INT8 那套 Orin 环境变量
公开延迟的含义	64GB、INT8、cache=1：约 124 ms。测的是默认 chunk 10 的 DROID	官方镜像 FP8 约 40.7 ms，NVFP4 约 20.3 ms。测的是 LIBERO、horizon 10

建议落地配置（两块板同一条策略，只是精度不同）

- 形状固定：2 路相机，chunk 32，去噪 10 步，prompt 上限 200，`state_prompt_mode=fixed`，CUDA graph 开，视觉层 27，不做 token 池化。
- Orin：BF16 对齐通过后，打开 `FVK_PI05_RTX_FORCE_INT8=1`，`cache_frames=1`。官方在 DROID 上这档与 BF16 余弦为 1.000。
- Thor：先 `use_fp8=False` 对齐，再 `use_fp8=True` 并用真实图像校准。NVFP4 放在 FP8 门禁之后，而且要确认 `pipe.Sa == 32`。
- Training-time RTC 的前缀长度固定为 6，和 HIL 的 `fixed_inference_delay=6` 一致。两块板的公开 `infer()` 都没有这个前缀，接机械臂之前要先改前端并通过第 8 节的门禁。

2. 这份权重的合同

目录：

```
checkpoints/outputs/pi05_policy_screw_sorting_single_sft_ep279_v30_annotated-pi07/checkpoints/030000/pretrained_model
```

拷到板上时这 7 个文件都要在：`config.json`、`model.safetensors`、`policy_preprocessor.json`、`policy_preprocessor_step_3_normalizer_processor.safetensors`、`policy_postprocessor.json`、`policy_postprocessor_step_0_unnormalizer_processor.safetensors`、`train_config.json`。

项	值	落到前端时
相机	`observation.images.high`，然后 `observation.images.right`。480×640，模型 224×224	`num_views=2`。high 送给 `image`，right 送给 `wrist_image`
状态 / 动作	各 7 维，内部 pad 到 32。夹爪是 `right_joint6.pos`	比较和后处理都只看前 7 维
动作语义	相对动作，夹爪排除在外，保持绝对	FlashRT 不做这一步。留在 LeRobot 的 `absolute_actions_processor`
归一化	state、action 都是分位数	两块板都会自动读上面的 processor safetensors
chunk / 步数	32 / 10	Orin 用参数；Thor 改 Sa
文本	tokenizer 最长 200。任务句见第 10 节	`fixed` 模式把 state 写进 prompt 后垫到固定长度，整段只捕获一张 graph
RTC	`rtc_training_config.enabled=true`，`max_delay=10`。`rtc_config` 为空	用训练时硬写入，不用推理时 guidance
精度	训练是 bfloat16，gemma_2b + gemma_300m	见第 4 节
后处理	反归一化 → 绝对动作 → OSQP（fps 30）→ cpu	OSQP 留在 LeRobot，不放进 CUDA graph

3. 两块板怎么编译

先确认板子。不是这两行之一就停。

```
nvidia-smi --query-gpu=name,compute_cap --format=csv
python -c "import torch; print(torch.cuda.get_device_capability())"
cat /etc/nv_tegra_release
```

Orin 必须打印 `(8, 7)`。Thor 必须打印 `(11, 0)`。锁最高功耗和固定时钟，后面的毫秒数才有意义：

```
sudo nvpmodel -m 0
sudo jetson_clocks
```

3.1 Orin

```
git clone https://github.com/flashrt-project/FlashRT.git
cd FlashRT
git clone --depth 1 --branch v4.4.2 https://github.com/NVIDIA/cutlass.git third_party/cutlass
python3 -m venv --system-site-packages .venv
source .venv/bin/activate
pip install -e ".[torch]"
export PATH=/usr/local/cuda/bin:$PATH
export CUDACXX=/usr/local/cuda/bin/nvcc
cmake -B build_orin_sm87 -S . \
  -DGPU_ARCH=87 \
  -DFA2_ARCH_NATIVE_ONLY=ON \
  -DFA2_HDIMS='96;128;256' \
  -DFA2_DTYPES='bf16' \
  -DPython3_EXECUTABLE="$(which python3)"
cmake --build build_orin_sm87 -j4
```

`-j4` 是为了避开 32GB 内存在全核编译时被换出。64GB 可以试 `-j6`，内存吃紧就回到 4。编完：

```
python -c "from flash_rt.hardware import detect_arch; print(detect_arch())"
```

输出必须是 `rtx_sm87`。

3.2 Thor

官方镜像 `ghcr.io/flashrt-project/flashrt-thor:thor-v0.2.0` 用来复现 LIBERO、horizon 10。这份螺丝模型是 horizon 32，不要拿这个镜像直接 `predict()`。在源码里改完再编。

打开 `flash_rt/frontends/torch/pi05_thor.py`，找到：

```
Sa, Da, Ha, La = 10, 1024, 4096, 18
```

只把 `Sa` 从 10 改成 32。`Da/Ha/La` 不动。文件里其余的 `steps = 10` 是去噪步数，保持 10。保存后搜索构造函数附近，确认没有第二处把动作长度又写成 10。然后：

```
git clone https://github.com/flashrt-project/FlashRT.git
cd FlashRT
python3 -m venv --system-site-packages .venv
source .venv/bin/activate
export PATH=/usr/local/cuda/bin:$PATH
export CUTE_DSL_ARCH=sm_101a
python -c "import torch; assert torch.cuda.get_device_capability()== (11, 0)"
pip install numpy==1.26.4 safetensors==0.8.0 sentencepiece pillow pybind11 ninja \
  nvidia-cutlass-dsl==4.5.1 quack-kernels==0.4.1
pip install --no-deps --no-build-isolation -e .
git clone --depth 1 --branch v4.4.2 https://github.com/NVIDIA/cutlass.git third_party/cutlass
cmake -S . -B build -DGPU_ARCH=110 -DCMAKE_CUDA_ARCHITECTURES=OFF \
  -DCMAKE_BUILD_TYPE=Release -DFLASHRT_ENABLE_PI05_THOR=ON
cmake --build build -j2 --target flash_rt_kernels flash_rt_fp4 fmha_fp16_strided flash_rt_pi05_thor
python -c "from flash_rt import flash_rt_kernels, flash_rt_fp4, flash_rt_pi05_thor; \
from flash_rt.hardware import detect_arch; print(detect_arch())"
```

输出必须是 `thor`。Thor 编译用 `-j2`，和官方指南一致。

PaliGemma 分词器：

```
mkdir -p "$HOME/.cache/openpi/big_vision"
curl -fL --retry 3 \
  https://storage.googleapis.com/big_vision/paligemma_tokenizer.model \
  -o "$HOME/.cache/openpi/big_vision/paligemma_tokenizer.model"
export FLASH_RT_PALIGEMMA_TOKENIZER="$HOME/.cache/openpi/big_vision/paligemma_tokenizer.model"
```

Orin 的 RTX 前端会优先用本机的 OpenPI `PaligemmaTokenizer`。板上若能装 openpi，两块板都设置同一个 `FLASH_RT_PALIGEMMA_TOKENIZER`，避免词表不一致。

4. 优化项对照

下表每一行都是一次独立实验。一次只改一行里的那一个旋钮，其余保持第 1 节的建议配置。公开毫秒数来自 chunk 10 的 DROID 或 LIBERO，不能当作这份 chunk 32 的验收线。验收线是第 7、8 节的余弦，再加上本机测到的中位数。

旋钮	Orin	Thor	改变了什么	对这份策略
BF16 / FP16 参考	`use_fp8=False`，不要设 INT8	`use_fp8=False`	权重和激活保持 16 位浮点	第一道门禁。后面的量化都和它比
W8A8	`FVK_PI05_RTX_FORCE_INT8=1`	没有这条路径	大矩阵的权重、激活都是 INT8，结果回到 BF16。Orin 部署文档在 DROID、cache=1 上写成与 BF16 逐位一致，延迟从约 193 ms 降到约 124 ms	Orin 的正式精度。本机余弦要自己再测
只量化编码器	`FVK_PI05_RTX_INT8_ENCODER_ONLY=1`	无	编码器 INT8，解码器留 BF16。解码器序列短，INT8 tile 在 16 个 SM 上经常更慢	全量 INT8 余弦掉了才退回这里
视觉 INT8	`FVK_PI05_RTX_INT8_VISION=1`	无	作者标明当时未做余弦验证。静态视觉 INT8 会把编码器余弦从 0.991 打到 0.282，代码里已关掉	不要开
FP8	SM 8.7 没有 FP8 tensor core，`use_fp8=True` 会被跳过	`use_fp8=True`，先 `calibrate`	权重和激活都是 FP8 E4M3，缩放来自真实图像。官方 LIBERO：约 40.7 ms，平均	Thor 的正式精度

			余弦 0.999905	
NVFP4	无	use_fp4=True 且 use_fp4_decoder=True，并带官方的 use_fa4=True、awq_alpha=0.5、encoder_p1_combiner="epilogue_hw_nod"、encoder_down_variant=8	4 位浮点，权重和激活都是 4 位。官方 LIBERO：约 20.3 ms，平均余弦 0.999707	FP8 门禁通过、并且 Sa==32 之后才测
cache_frames=1	默认。每帧都跑完整编码器	Thor 前端没有这个参数	Orin 文档写明与 BF16 路径逐位一致	Orin 落地用这个
cache_frames=2	偶数帧复用上一帧编码器 K/V	无	DROID 上有效约 12.2 Hz，余弦 0.991。两帧的耗时不一样（约 127 ms 和 39 ms），推理延迟一帧一个数	先不要和 d=6 的静态 RTC 图叠在一起。RTC 门禁通过之后再单开一轮
vision_pool_factor	1、2 或 4	不在 Thor 构造函数里	2 或 4 会把视觉 token 做 2×2 / 4×4 池化，序列变短，网络和训练时不同	保持 1
vision_num_layers	1 到 27	Thor 按 27 层 SigLIP 编进图	小于 27 等于拆掉后面的视觉层	保持 27
num_steps	构造参数，这份模型是 10	源码里 steps = 10	改步数就是另一条 ODE	保持 10
state_prompt_mode	exact 按当次 prompt 长度各捕获一张图；fixed 垫到固定长度，一张图	同左，另有 state_prompt_fixed_max_len	state 离散化之后写进 prompt，长度会变。exact 在控制循环里反复重建图	用 fixed，上限 200
CUDA graph	use_cuda_graph=True	同左。autotune=3 是默认试跑次数	捕获之后重放不再走	测速和上机都

			Python 调度。关掉只为了用 nsys 看单个 kernel	开着。nsys 时临时关掉
Thor 的 `rtc_prefix` 图	无	按 `prefix_len` 另捕获一张解码图，缓冲名叫 `rtc_guidance_weight`	这是推理时 guidance RTC。本权重的 `rtc_config` 是空的	不要调用。训练时 RTC 按第 8 节另做

5. 构造模型

`load_model()` 不转发 `chunk_size` 和 `max_prompt_len`。Orin 直接构造 RTX 类。Thor 在改完 `Sa` 之后构造 Thor 类，不要用官方示例里 horizon 10 的 `load_model` 片段当作这份权重的入口。

5.1 Orin，BF16 参考

```
from flash_rt.frontends.torch.pi05_rtx import Pi05TorchFrontendRtx

CKPT = "/abs/path/to/pretrained_model"
pipe = Pi05TorchFrontendRtx(
    CKPT,
    num_views=2,
    chunk_size=32,
    num_steps=10,
    max_prompt_len=200,
    vision_pool_factor=1,
    vision_num_layers=27,
    cache_frames=1,
    state_prompt_mode="fixed",
    use_fp8=False,
    use_cuda_graph=True,
    denoise_trace=True,
)
assert pipe.chunk_size == 32 and pipe._num_steps == 10
```

5.2 Orin，W8A8

环境变量要在导入前端之前设置。

```
import os
os.environ["FVK_PI05_RTX_FORCE_INT8"] = "1"
# 然后用与 5.1 相同的参数再构造一次
```

5.3 Thor

```
from flash_rt.frontends.torch.pi05_thor import Pi05TorchFrontendThor

pipe = Pi05TorchFrontendThor(
    CKPT,
    num_views=2,
    use_cuda_graph=True,
    autotune=3,
    use_fp8=False,          # 第一轮对齐。通过后改为 True
    use_fa4=True,
    state_prompt_mode="fixed",
    state_prompt_fixed_max_len=200,
)
print("Sa", pipe.Sa)
assert pipe.Sa == 32
```

`Sa` 若仍是 10，说明改动没有进到正在导入的那份源码，停。FP8 轮把 `use_fp8=True`，在 `set_prompt` 之后、进控制循环之前调用：

```
pipe.calibrate(real_observations, percentile=99.9)
```

`real_observations` 用现场的 high / right 图像，至少 8 帧，覆盖盒子在画面里的正常位置。校准会按这些帧的激活统计写 FP8 缩放并重新捕获 graph。换场景之后重新校准。

6. 输入输出约定

图像用 224×224、HWC、uint8，范围 0–255，交给前端自己做 `/127.5 - 1`。不要先用 LeRobot 的视觉归一化再送进去，否则会归一化两次。相机顺序是 high，然后 right。

Prompt 与 HIL 脚本相同：

```
Please sort and return the silver screws in the grey box to their proper places.
```

状态 7 维，顺序与数据集 feature 名一致，夹爪在最后。前端把离散化后的状态写进 prompt。

噪声形状 `(32, 32)`，后 25 维是 pad。对齐时两边喂同一份噪声。

	Orin infer 的返回值	**Thor infer 的返回值**
已反归一化的 7 维	`actions`，形状 (32, 7)	`actions`，形状 (32, 7)
归一化的 32 维	`denoise_trace=True` 时的 `raw_actions`，形状 (32, 32)	在返回前存在 `_g_noise`，默认不放进返回值。对齐时用 `debug=True` 的日志，或在 `unnormalize_actions` 之前把 `raw_actions` 存下来

接入 LeRobot 时：

- Orin 取 `raw_actions[:, :7]`，还原成 `(1, 32, 7)`，走完整后处理：分位数反归一化 → 绝对动作（夹爪保持绝对）→ OSQP → cpu。
- Thor 的 `actions` 已经做过反归一化。再跑一遍 unnormalizer 会把量纲乘错。只跑绝对动作和 OSQP。

7. 第一帧对齐（没有 RTC）

第一帧没有上一 chunk，LeRobot 的 `sample_actions` 走普通 10 步去噪。准备一条离线样本：两张 uint8 图、7 维状态、同一句 prompt、同一份噪声。

LeRobot 侧：`inference_delay=None`，`prev_chunk_left_over=None`，`num_steps=10`。取出反归一化之后、绝对动作之前的 `(32, 7)`。

FlashRT 侧：`set_prompt` 一次，再 `infer`。Orin 用反归一化后的 `actions`。Thor 同样用 `actions`。比较这 32×7。

门禁，BF16 参考：

- 最大绝对误差 < 1e-2。
- 余弦 > 0.999。
- 32 个数都是有限值。

不过就只查这四项：`chunk` 是不是 32、`steps` 是不是 10、相机是不是 high 在前、prompt 有没有被截到 48（Orin 的类默认值）。Thor 再查一次 `pipe.Sa`。

BF16 通过之后再测正式精度，样本和噪声不变：

板	配置	相对 BF16 参考的门禁
Orin	INT8，cache_frames=1	余弦 > 0.999。官方 DROID 写的是 1.000。本机若低于 0.999，退回 BF16 上机
Thor	FP8，已用现场图像校准	平均余弦 ⩾ 0.999，最差样本 ⩾ 0.995。这是官方 LIBERO 的验收线，沿用到这份权重
Thor	NVFP4	同上，而且 `Sa==32`。低于 0.995 就固定用 FP8

测速丢掉前 200 次（含校准和捕获），再取 1000 次的中位数和 p95。把功耗模式、时钟、`Sa`、精度写进第 11 节的表。Orin 的 124 ms、Thor 的 20–41 ms 只作为数量级，不作为失败条件。

8. Training-time RTC

本权重每一步去噪做两件事，以 `src/lerobot/policies/pi05/modeling_pi05.py` 的 `sample_actions` 为准：

1. 算速度之前，把 `x_t` 的前 6 步写成上一 chunk 的剩余动作。这 6 个位置的 flow time 是 0，后 26 个位置用当前步的 time。`dt = -1/10`，time 从 1.0 走到 0.1。
2. Euler 更新 `x_t = x_t + dt * v_t` 之后，再把前 6 步写回同一段剩余动作。

第一帧 `prev_chunk_left_over is None`，不钉。从第二帧起钉前 6 步。前缀在归一化空间，pad 维是 0。不要把后处理之后的绝对关节角喂回去。

8.1 Orin

`Pi05TorchFrontendRtx._precompute_decoder_styles` 现在把一个 time 铺满 32 个位置。改成静态的两段，这样 CUDA graph 仍然只有一张：

- 对 `step = 0 .. 9`，`t_suffix = 1.0 + step * (-1/10)`。
- 位置 `0:6` 用 `t = 0` 的 time MLP 输出。
- 位置 `6:32` 用 `t_suffix` 的输出。

- `time_emb`、`style_attn`、`style_ffn`、`style_final` 都按位置写，不再 expand 成整段同一个向量。

去噪循环里加两次固定长度的拷贝，放进已捕获的 graph：每个 step 算速度之前，以及 Euler 之后，执行 `x_t[:6, :] = prefix[:6, :]`。`infer` 增加 `prev_action_norm`，形状 `(32, 32)`。第一帧传 `None`，走第 7 节那张普通图；之后传剩余动作，走这张 d=6 的图。两张图都是静态的，不要为 0–10 的延迟各捕获一张。

8.2 Thor

Thor 的 `infer()` 不读上一 chunk。类里已有的 `_capture_model_runtime_rtc_prefix_graph` 使用 `rtc_guidance_weight`，那是推理时 guidance，和这份权重的训练方式不是同一条。不要把 `prefix_len=6` 配上 guidance 权重当作训练时 RTC。

要改的是 Thor 解码循环里的 time 和 `x_t`，语义与 8.1 相同：前 6 个位置 time 为 0，Euler 前后各写回一次前缀，`Sa` 已经是 32。捕获一张 `prefix_len=6` 的静态图。图的输入缓冲是归一化剩余动作，不是 guidance 标量。

8.3 第二帧门禁

1. 第一帧两边都不带前缀，保存 LeRobot 的归一化 chunk，记为 `prev`，形状 `(32, 32)`。
2. 第二帧换图、换噪声。LeRobot 调用 `sample_actions(..., inference_delay=6, prev_chunk_left_over=prev)`。FlashRT 传入同一份 `prev`。
3. 比较反归一化之后、绝对动作之前的 `(32, 7)`。

门禁：

- 前 6 步与 `prev` 反归一化后的前 7 维最大绝对误差 < 1e-5。
- 后 26 步最大绝对误差 < 1e-2，余弦 > 0.999（BF16）。量化档再相对这条 BF16+RTC 结果：Orin INT8 余弦 > 0.999，Thor FP8 / NVFP4 平均余弦 ⩾ 0.999、最差 ⩾ 0.995。

前 6 步对不上，是前缀没写进 `x_t`，或写进之后被 Euler 冲掉。后 26 步对不上而前 6 步是对的，是 time 仍然整段相同。两种都回到 8.1 或 8.2。

9. 接到 HIL

只替换 `examples/rac/hil_data_collection.py` 推理线程里 `predict_action_chunk` 所进入的去噪。队列、`ActionQueue.merge`、`fixed_inference_delay=6`、`execution_horizon=10`、`action_queue_size_to_get_new_actions=12` 不动。

`scripts/run_hil_collect_data.sh` 的 spacemouse 分支改成：

```
--use_torch_compile=false
--fixed_inference_delay=6
--rtc.execution_horizon=10
--action_queue_size_to_get_new_actions=12
--dataset.fps=30
```

`rtc.max_guidance_weight` 在训练时 RTC 打开时不会进入去噪，留着即可。不要把它改成推理时 guidance 来配合 FlashRT。

数据流：

1. LeRobot preprocessor 继续做相机顺序、resize 到 224、分位数归一化、状态写入 prompt。送给 FlashRT 的图像用 resize 之后的 uint8，让前端做自己的 `/127.5-1`。
2. 第一帧 `prev_action_norm=None`。之后把队列里尚未执行的归一化动作按时间对齐，长度不足 6 的用上一 chunk 里已经对齐的剩余补齐，pad 维填 0。
3. Orin 把 `raw_actions[:, :7]` 送进完整后处理。Thor 把已反归一化的 `actions` 送进绝对动作和 OSQP。
4. 后处理后的 chunk 仍交给原来的 `merge`。

控制周期是 30 Hz，一步 33 ms。chunk 32、阈值 12 时，两次推理之间大约执行 20 步，队列里留下约 12 步，也就是大约 400 ms。推理中位数要低于 400 ms，队列才不会被抽空。训练允许的最大延迟是 10 步（约 333 ms），`fixed_inference_delay=6`（约 200 ms）盖住 Orin INT8 大约 4 步、Thor FP8 大约 1 到 2 步的实际耗时。测完之后如果中位数稳定地只占 1 到 2 步，可以再把 d 改小，但要连同第 8 节的静态图一起改，并重做第二帧门禁。

Orin 32GB 的公开 BF16 大约 216 ms，仍低于 400 ms，可以跑。优先用 64GB 跑 INT8，显存里同时放得下 graph 和校准缓冲。Thor 的统一内存更宽裕，限制在 `Sa=32` 的图能不能捕获，不在权重装不装得下。

10. 上机

空跑一个 episode，手不要放在示教器上。任务句：

```
Please sort and return the silver screws in the grey box to their proper places.
```

日志里应看到：第一次推理没有前缀，之后每次延迟是 6；`delayqueue` 在一次推理之后降到 12 附近再触发下一次，而不是停在 28 附近。停在 28 说明几乎只发出了被钉住的前缀，手臂会停在当前关节。那是队列阈值，阈值保持 12。

记录里 `|action - state|` 的均值应明显高于 0.001 rad。回到 0.001 附近时，先看阈值是不是 12，再看传给 FlashRT 的前缀是不是上一 chunk 的真实剩余。

同时满足才算落地：

1. 第 7 节第一帧门禁通过。
2. 第 8 节前 6 步被钉住，后 26 步过余弦门禁。
3. 正式精度（Orin INT8 或 Thor FP8）相对 BF16 的门禁通过。NVFP4 未过就不要上机。
4. 一个 episode 里队列会消费到阈值 12，手臂离开当前关节。
5. 后处理仍是绝对动作加 OSQP。干预数据仍由原来的 spacemouse 分支写入。

11. 现场记录表

项	Orin	Thor
板型 / 内存		
`detect_arch()`		
chunk / Sa		
精度		
nvpmodel / jetson_clocks		
第一帧最大误差 / 余弦		
第二帧前 6 步误差		
第二帧后 26 步余弦		
量化相对 BF16 的余弦		
200 次预热后的中位数 / p95（ms）		
空跑时 delayqueue 最低值		
`|action-state|` 均值（rad）		

12. 停手条件

- 计算能力不是 8.7 或 11.0。
- Thor 上 `pipe.Sa` 不是 32。
- 用官方 `load_model(...).predict()` 直接驱动这份 chunk 32 的权重。
- 在 Orin 上设置 `use_fp8=True` 或 NVFP4，指望它生效。
- 打开 `FVK_PI05_RTX_INT8_VISION`、`vision_pool_factor` 不为 1，或 `vision_num_layers` 小于 27。
- 把 Thor 的 `rtc_guidance_weight` 图当作训练时 RTC。
- 在 FlashRT 去噪之外再套 `torch.compile`。
- 把 FlashRT 已经反归一化的 7 维再送进 LeRobot unnormalizer，或把 Orin 的归一化 `raw_actions` 直接发给关节。
- 为了延迟把去噪步数改成不是 10，或把 chunk 改成不是 32。
- 单独修改 `fixed_inference_delay`，而第 8 节的静态图仍是另一个 d。

公开数字出处：FlashRT `docs/deployment_orin.md`（Orin 64GB，INT8，cache 1/2，chunk 为前端默认 10），`docs/thor/pi05.md`（Thor 官方镜像，LIBERO，horizon 10，FP8 40.71 ms，NVFP4 20.33 ms）。本手册的验收以第 7、8 节在这份 030000 权重上的测量为准。